AUTOM-08: Migration Automation

> **Series:** AUTOM — Dynatrace Automation | **Notebook:** 8 of 9 | **Created:** January 2026 | **Last Updated:** 10/22/2026

Configuration migration is the process of transferring Dynatrace settings from one environment to another. This is common in tenant consolidation, Managed-to-SaaS migration, and disaster recovery scenarios.

Table of Contents

1. Introduction
2. Migration Scenarios
3. Monaco Migration
4. Terraform Export
5. SaaS Upgrade Assistant
6. Validation and Verification
7. Summary

Prerequisites

Before starting this notebook, ensure you have:

Requirement	Description
Source tenant access	Admin access to source environment
Target tenant access	Admin access to target environment

Requirement	Description
Credentials	On both source and target: an access token **and** a platform token (or OAuth client) — platform types (SLOs, workflows, documents, segments) need the latter
Monaco or Terraform	Migration tool installed

Learning Objectives

By the end of this notebook, you will:

- Understand common migration scenarios
- Know how to export and import configurations
- Be able to handle non-portable configurations
- Validate migration completeness

---

1. Introduction

Common Migration Scenarios

Scenario	Description
Managed to SaaS	Moving from self-hosted to Dynatrace SaaS
Tenant Consolidation	Merging multiple tenants into one
Environment Cloning	Creating dev/staging from production
Disaster Recovery	Restoring config to a new tenant
Config Backup	Periodic export for safekeeping

Where the Effort Goes

In community practice, most configuration migrates with tooling, and the remainder — credentials, integrations, entity references — takes most of the effort.

Plan extra time for:
- Credentials and secrets
- Custom integrations
- Entity ID references
- Network-specific settings

---

2. Migration Scenarios

What Can Be Migrated

Configuration Type	Portable	Notes
Segments	Yes	Filter trees migrate; deploy before anything that references them
Workflows	Partial	Portable, but actors, connections and credentials are tenant-specific — re-point them
Dashboards and notebooks (documents)	Partial	Entity IDs and segment IDs inside queries need updating
SLOs — modern (`slo-v2`)	Yes	DQL SLI; replace entity IDs with tags, and reference segments through Monaco rather than by ID
Davis anomaly detectors	Yes	DQL query; the `actor` service user is tenant-specific
Classic: Management Zones	Yes, to a classic target	Rules migrate, entity IDs don't — blocked at upgrade, so migrate the *successor* (segments + IAM) into a Latest Dynatrace target

Configuration Type	Portable	Notes
Classic: Auto-tagging Rules	Yes, to a classic target	Blocked at upgrade — see FAQ-02 for the primary-tags successor
Classic: Alerting Profiles	Yes, to a classic target	May need zone ID updates — blocked at upgrade; successor is a problem-triggered workflow
Classic: Dashboards	Partial	Entity IDs need updating
Classic: SLOs (`builtin:monitoring.slo`)	Yes, to a classic target	Metric expressions portable — blocked at upgrade; rewrite as `slo-v2` (SLO-05)
Synthetic Monitors	Partial	Location IDs may differ
Request Attributes	Yes	Fully portable
Calculated Services	Yes	Fully portable

What Cannot Be Migrated

Configuration Type	Why Not	Action Required
Credentials Vault	Security isolation	Re-enter manually
API Tokens	Tenant-specific	Generate new tokens
Historic Data	Not transferable	Accept baseline gap
Cloud Integrations	Credential-dependent	Reconfigure
SSL Certificates	Tenant-specific	Re-upload
Private Locations	Infrastructure-specific	Redeploy

Migration Tool Comparison

Tool	Best For	Pros	Cons
Monaco	Config-as-code teams	YAML-based, Git-friendly	Manual entity ID handling
Terraform Export	IaC environments	Full HCL output	Large state files
SaaS Upgrade Assistant	M2S migration	Guided, UI-based	Limited to M2S
Settings API	Custom scripts	Full control	Most manual effort

---

3. Monaco Migration

Step 1: Download from Source

```
# Set source environment
export DT_SOURCE_URL="https://source-tenant.live.dynatrace.com"
export DT_SOURCE_TOKEN="&lt;your-source-api-token&gt;"
export DT_SOURCE_PLATFORM_TOKEN="&lt;your-source-platform-token&gt;"

# Download all configurations — settings, classic APIs AND platform types
# --token / --platform-token take the NAME of the variable holding the token,
not the token itself
monaco download \
  --url "$DT_SOURCE_URL" \
  --token DT_SOURCE_TOKEN \
  --platform-token DT_SOURCE_PLATFORM_TOKEN \
  --output-folder ./migration-export
```

The platform token is what brings SLOs (`slo-v2`), workflows, documents, Grail buckets and segments into the export — the `--oauth-client-id` / `--oauth-client-secret` pair is the alternative. **Drop it and the export skips every platform type**: an access token alone does not reach the Platform APIs. Monaco prints one `WARN` line per skipped type (`Skipped downloading … due to missing OAuth credentials`) and the download still succeeds, so the gap is easy to miss in a long log. Only a download that requests a *single* platform type with one `--only-*` flag fails outright.

The download writes a `manifest.yaml` at the top of `migration-export/` and the configs into a `project/` sub-folder (the default `--project` name).

Sources: download_configs.go @ v2.30.0 (Dynatrace GitHub), download_command.go @ v2.30.0 (Dynatrace GitHub).

Step 2: Review and Clean

After download, review the exported configs:

```
# List exported configuration types
ls -la migration-export/

# Count configurations per type
find migration-export -name "config.yaml" | wc -l
```

Configuration Cleanup

Remove or update:

Item	Action
Entity IDs	Replace with selectors or names
Location IDs	Map to target locations
Credential references	Update to target vault entries
Environment-specific URLs	Update endpoints

Step 3: Create Target Manifest

manifest.yaml:

```
manifestVersion: 1.0

projects:
  - name: migration
    path: migration-export/project   # the configs — not the download root,
which also holds a manifest.yaml

environmentGroups:
  - name: default
    environments:
      - name: target
        url:
          type: environment
          value: DT_TARGET_URL
        auth:
          token:
            name: DT_TARGET_TOKEN
          platformToken:
            name: DT_TARGET_PLATFORM_TOKEN
```

Keep this manifest **outside** `migration-export/`. The project path points at the `project/` sub-folder the download created; pointing it at the download root makes Monaco read the downloaded `manifest.yaml` as if it were a config file.

The target needs **both** credentials. Without `platformToken` (or an `oAuth` client), Monaco refuses to deploy the platform configs the download brought along — `slo-v2`, workflows, documents, segments and buckets — with *"requires platform credentials"*, and `--dry-run` fails the same way.

Sources: Manage resources — auth section (DT docs) — *"Access tokens and platform tokens are not interchangeable."*; deploy.go @ v2.30.0 (Dynatrace GitHub).

Step 4: Validate and Deploy

```
# Set target environment
export DT_TARGET_URL="https://target-tenant.live.dynatrace.com"
export DT_TARGET_TOKEN="&lt;your-target-api-token&gt;"
export DT_TARGET_PLATFORM_TOKEN="&lt;your-target-platform-token&gt;"

# Dry run — Monaco has no separate "validate" command; this parses the YAML,
# checks template JSON and resolves references without contacting the tenant
monaco deploy manifest.yaml --environment target --dry-run

# Deploy
monaco deploy manifest.yaml --environment target
```

Handling Entity ID References

Dashboards and alerting profiles often contain entity IDs. These need special handling:

Option 1: Use an entity selector — only where the schema has one

Some configuration schemas accept an `entitySelector` field instead of a fixed ID. Where the target schema defines one, a selector survives the move:

```
// Before (hardcoded ID)
{
  "entityId": "HOST-ABC123DEF456"
}

// After (selector)
{
  "entitySelector": "type(HOST),entityName(\"web-server-01\")"
}
```

This is not a generic transform. Renaming the key in a payload whose schema has no `entitySelector` field fails at deploy with an HTTP 400, and `monaco deploy --dry-run` does not catch it, because the dry run does not validate payload content. Everywhere else, map IDs (Option 2) or, in Monaco, parameterise the ID per environment.

Option 2: Entity Mapping Script

```
import json
import re

def replace_entity_ids(config: dict, mapping: dict) -&gt; dict:
    """Replace entity IDs using a source-&gt;target mapping."""
    config_str = json.dumps(config)

    for source_id, target_id in mapping.items():
        config_str = config_str.replace(source_id, target_id)

    return json.loads(config_str)

# Build mapping from entity names
entity_mapping = {
    "HOST-ABC123": "HOST-XYZ789",
    "SERVICE-DEF456": "SERVICE-UVW012"
}
```

---

4. Terraform Export

Using the Export Utility

The export utility is **not a separate download** — it is the provider binary itself, invoked with `-export`. After `terraform init` the executable sits under `.terraform/providers/registry.terraform.io/dynatrace-oss/dynatrace/<version>/<os_arch>/`.

```
# Linux / macOS
./terraform-provider-dynatrace -export [-ref] [-migrate] [-import-state] [-id]
[-flat] [-exclude] [&lt;resourcename&gt;[=&lt;id&gt;]]

# Windows
terraform-provider-dynatrace.exe -export [options] [&lt;resourcename&gt;
[=&lt;id&gt;]]
```

Credentials are supplied through **environment variables**, not command-line arguments:

Variable	Required	Purpose
`DYNATRACE_ENV_URL`	Yes	Source tenant endpoint

Variable	Required	Purpose
`DYNATRACE_API_TOKEN`	Yes	API token for the source tenant
`DT_CLIENT_ID` / `DT_CLIENT_SECRET` / `DT_ACCOUNT_ID` (OAuth client), or `DYNATRACE_PLATFORM_TOKEN`	For platform resources	Workflows, segments, documents and platform SLOs are read through the platform APIs, which an API token alone does not reach
`DYNATRACE_TARGET_FOLDER`	No	Output directory (default: `./configuration`)

```
export DYNATRACE_ENV_URL="https://source-tenant.live.dynatrace.com"
export DYNATRACE_API_TOKEN="&lt;your-source-api-token&gt;"
export DYNATRACE_TARGET_FOLDER="./terraform-export"

./terraform-provider-dynatrace -export -ref -id
```

> **Never pass the token as a command-line argument.** Anything on the command line is visible to every user who can run `ps` on that host, and it lands in shell history and CI job logs. The utility reads credentials from the environment for exactly this reason. AUTOM-04 §3 covers the provider auth model in full.

Migrating Between Tenants — the `-migrate` Flag

For tenant-to-tenant moves (Managed to SaaS, or consolidation) use `-migrate` rather than `-ref`. The two are mutually exclusive: `-ref` emits data-source references for a tenant you will keep managing in place, while `-migrate` resolves dependencies for recreating configuration in a *different* tenant.

Approach	Command	Use when
Bulk	`./terraform-provider-dynatrace -export -migrate`	Target tenant is fresh, with no configuration to preserve
Iterative	`./terraform-provider-dynatrace -export -migrate -datasources <resourcename>`	Target already holds configuration you must not overwrite

Export with the environment pointed at the source, then repoint it at the target to apply. The migration guide also documents source-scoped variables

(`DYNATRACE_SOURCE_ENV_URL` , `DYNATRACE_SOURCE_API_TOKEN`) so both endpoints can be held at once.

> **Entity IDs do not survive the move.** A migrated OneAgent can register as a *new* entity in the target, which silently breaks any configuration pinned to an entity ID. Re-check everything that references a specific entity after apply — this is the single most common post-migration surprise.

Export Output Structure

Output defaults to a **module structure** — one directory per resource family — written to `DYNATRACE_TARGET_FOLDER` , or `./configuration` if unset. Pass `-flat` to put everything in a single directory instead.

```
configuration/
├── main.tf
├── providers.tf
├── dynatrace_automation_workflow/   # only when named explicitly — excluded
by default
│   └── *.tf
├── dynatrace_segment/              # only when named explicitly — excluded by
default
│   └── *.tf
├── dynatrace_document/             # only when named explicitly — excluded by
default
│   └── *.tf
├── dynatrace_management_zone_v2/   # classic — present only while the source
still has them
│   └── *.tf
├── .flawed/              # deprecated configs requiring modification before
apply
└── .requires_attention/  # items missing essentials (e.g. credential payloads
the API cannot return)
```

Triage `.flawed/` and `.requires_attention/` before committing anything — the second directory is where secrets that the source API refuses to return end up, and they must be re-entered by hand.

Dashboards, workflows, segments, documents and platform SLOs are all excluded by default. A bulk `-export -migrate` leaves them behind without an error, so a migration that stops there loses them. Export them in a separate run that names them (naming resources limits the export to those resources), with the platform credentials above set:

```
DYNATRACE_TARGET_FOLDER=./terraform-export-platform \
  ./terraform-provider-dynatrace -export -migrate \
  dynatrace_automation_workflow dynatrace_segment dynatrace_document
dynatrace_platform_slo
```

Run `-list-exclusions` to see the full default-exclusion list, and add anything else on it that you need.

Importing to the Target

```
cd configuration

# Repoint the provider at the target tenant
export DYNATRACE_ENV_URL="https://target-tenant.apps.dynatrace.com"
export DYNATRACE_API_TOKEN="&lt;your-target-api-token&gt;"

terraform init
terraform plan     # expect a large first plan — review it before applying
terraform apply
```

Adding `-import-state` to the original export runs `terraform init` and imports the exported resources into state automatically, if you want a bootstrapped workspace rather than HCL files alone.

The full `-export` flag reference lives in **AUTOM-04 §8**. The end-to-end Managed-to-SaaS Terraform walkthrough — export, triage, repoint, apply, verify — is in **M2S-95 LAB**, and is not repeated here.

Sources: Terraform export utility (DT docs) — *"These files are moved to .requires_attention"*; Terraform migration guide (DT docs); automation_workflow resource (Dynatrace GitHub) — *"This resource is excluded by default in the export utility"* (the `segment`, `document` and `platform_slo` pages say the same); Provider configuration (Dynatrace GitHub) — *"The Dynatrace platform token used for platform APIs."*

---

5. SaaS Upgrade Assistant

For Managed-to-SaaS migrations, Dynatrace provides a guided tool.

Accessing the Assistant

The Assistant is an app in the **target SaaS environment**, fed by an export taken on the Managed side:

1. **Install the app.** In the target SaaS environment, open **Dynatrace Hub**, select **SaaS Upgrade Assistant**, then **Install**.
2. **Export from Managed.** Sign in to the Managed **Cluster Management Console**, go to **Environments**, select the environment to migrate from, and select **Export configuration**. Store the archive locally.
3. **Upload the archive in the app** and work through the imported configurations.

Upload the archive the Cluster Management Console produces. The docs describe no manual repackaging step and no archive layout to build by hand. They do recommend exporting from a Managed cluster on the same major version as the SaaS environment, to avoid false-positive failed configurations.

What the Assistant Does

Capability	Description
Progress tracking	Track configuration migration progress
Review failures	Browse imported configurations; failed ones are marked in red, with error messages for failed and skipped configurations
Edit and bulk edit	Fix one configuration in an edit form, or update hundreds at once in bulk mode, with a change preview
Partial deployment	Choose which configurations to migrate and deploy only those
Dashboard owners	Automatically update dashboard owners

Sources: SaaS Upgrade Assistant (DT docs) — *"Choose which configurations you want to migrate and run partial deployment."*; Migrate configuration (DT docs) — *"you export the Dynatrace Managed environment's configuration in the Cluster Management Console and upload it to the app."*

When to Use

Dynatrace sanctions **three** approaches for Managed-to-SaaS configuration migration. The Assistant is the recommended default, but it is not the only supported path — if you already run config-as-code, use the tool you already run.

Scenario	Recommended Tool	Notes
Managed to SaaS — no config-as-code today	**SaaS Upgrade Assistant**	Recommended default. Guided UI, progress tracking, bulk edit, partial deployment.
Managed to SaaS — already running Monaco	**Monaco**	Download from Managed, deploy to SaaS with a retargeted manifest (§3 above).
Managed to SaaS — already running Terraform	**Terraform** (`-export -migrate`)	Repoint the provider at the SaaS tenant. Full walkthrough in **M2S-95 LAB**.
SaaS to SaaS	Monaco	
Backup/Restore	Monaco or Terraform	
GitOps workflow	Monaco or Terraform	

Whichever path you pick, one set of settings **never** migrates automatically and must be recreated by hand: extension and cloud credential configurations (AWS, Azure, GCP, Cloud Foundry, Kubernetes), access tokens and personal access tokens, problem-notification integrations (Jira, OpsGenie, PagerDuty, and the rest), mobile symbolication and JavaScript error settings, request naming and merged services, multi-dimensional analysis saved views, account management (users, groups, permissions), tags and custom entity names, and process-grouping rules — which must be migrated *before* the upgrade, not after. Budget for this explicitly: it is the remainder from §1 that takes most of the effort.

Source: Migrate configuration (DT docs).

---

6. Validation and Verification

Pre-Migration Checklist

Check	Description
[ ] Source inventory	Document all configurations
[ ] Credential list	Identify all secrets to re-enter
[ ] Entity dependencies	Map entity ID references
[ ] Network requirements	Verify target connectivity
[ ] Token scopes	Ensure sufficient permissions

Post-Migration Validation

Run these DQL queries on the target tenant to verify entity counts:

> **Note:** `fetch dt.settings` is NOT a valid DQL data object. Settings objects must be queried via the Settings API (`GET /api/v2/settings/objects`), not DQL. The cells below document the correct approach.

```
// Count management zones
// NOTE: fetch dt.settings is NOT a valid DQL data object.
// Settings objects cannot be queried via DQL.
// Use the Settings API instead:
//   GET /api/v2/settings/objects?schemaIds=&lt;schemaId&gt;
// Example:
//   GET /api/v2/settings/objects?schemaIds=builtin:management-zones
//   GET /api/v2/settings/objects?schemaIds=builtin:tags.auto-tagging
```

```
// Count auto-tagging rules
// NOTE: fetch dt.settings is NOT a valid DQL data object.
// Settings objects cannot be queried via DQL.
// Use the Settings API instead:
//   GET /api/v2/settings/objects?schemaIds=&lt;schemaId&gt;
// Example:
//   GET /api/v2/settings/objects?schemaIds=builtin:management-zones
//   GET /api/v2/settings/objects?schemaIds=builtin:tags.auto-tagging
```

```
// Verify synthetic monitors — all three monitor types, Smartscape form
(preferred for new queries)
smartscapeNodes "BROWSER_MONITOR", "HTTP_MONITOR",
"NETWORK_AVAILABILITY_MONITOR"
| summarize total = count(), by:{type}

// Counting BROWSER_MONITOR alone misses HTTP and network-availability
monitors, so a
// migration check could pass with half the monitors missing. Compare per
type.
// A type with no monitors produces NO row (not a 0) — treat a missing row as
0 when
// comparing source and target.
//
// Classic form — still functional, and a genuine fallback (one query per
type):
//   fetch dt.entity.synthetic_test        // browser
//   | summarize total = count()
//
// dt.entity.synthetic_test maps to the BROWSER_MONITOR Smartscape node type.
Both forms
// returned the same monitor count when live-verified 07/30/2026, so either
works today;
// dt.entity.* is deprecated, so prefer the Smartscape form for anything new.
Note that
// dt.entity.* is a lookback view: with a wide window (from:-30d) it also
counts monitors
// deleted inside that window, while smartscapeNodes counts current nodes (re-
verified 09/18/2026).
//
// Two things to know before porting a synthetic query:
//   - Clickpath steps are a SEPARATE node type (BROWSER_MONITOR_STEP), not
fields of the
//     monitor. A query that reads per-step data must traverse to those nodes
rather than
//     expect step attributes on the monitor node.
//   - The other synthetic types map too: dt.entity.http_check -&gt;
HTTP_MONITOR,
//     dt.entity.multiprotocol_monitor -&gt; NETWORK_AVAILABILITY_MONITOR,
//     dt.entity.synthetic_location -&gt; SYNTHETIC_LOCATION.
```

Validation Script

Compare source and target configuration counts. Do it by **config type across two Monaco downloads** — one from each tenant, with the same flags — so platform types

(SLOs, segments, workflows, documents) are counted the same way as Settings 2.0:

```
from collections import Counter
from pathlib import Path

import yaml  # pip install pyyaml


def count_configs(download_root: Path) -&gt; Counter:
    """Count configs per Monaco config-type folder in a `monaco download`
output."""
    counts: Counter = Counter()
    for config_file in download_root.rglob("config.yaml"):
        doc = yaml.safe_load(config_file.read_text()) or {}
        counts[config_file.parent.name] += len(doc.get("configs", []))
    return counts


def validate_migration(source_export: Path, target_export: Path) -&gt;
list[dict]:
    """Compare per-type config counts between two downloads."""
    source, target = count_configs(source_export),
count_configs(target_export)
    return [
        {"type": t, "source": source[t], "target": target[t], "match":
source[t] == target[t]}
        for t in sorted(set(source) | set(target))
    ]


# Both downloads must use the same credentials shape: a download without a
platform
# token has no slo-v2 / segment / workflow / document folders at all, and
would
# "match" on zero for every platform type.
for row in validate_migration(Path("source-export"), Path("target-export")):
    print(row)
```

A type that appears on only one side shows up with a zero on the other, rather than being skipped — which is the failure the classic script below could not see.

Classic — Settings API count, for classic schemas only:

> **Two things to know before running this.** The SLO schema is `builtin:monitoring.slo` — **not** `builtin:slo`, which does not exist. Querying a nonexistent schema returns `totalCount: 0` rather than an error, so a count-

> comparison script using the wrong ID reports `0 == 0` and a cheerful match for a domain it never actually checked. This script previously carried that bug.
>
> All four schemas below are also marked **Blocked at upgrade** in AUTOM-02's catalog. After a tenant upgrades to the latest Dynatrace they return zero on both sides, and this script will again report a clean match — for configuration that no longer exists. Count-parity validation is only meaningful while both tenants are on the same generation; across a Classic → Gen3 boundary you need to compare the *replacement* constructs instead.

```
import requests

def count_settings(url: str, token: str, schema_id: str) -&gt; int:
    """Count settings objects of a given schema."""
    response = requests.get(
        f"{url}/api/v2/settings/objects",
        params={"schemaIds": schema_id},
        headers={"Authorization": f"Api-Token {token}"}
    )
    return response.json().get("totalCount", 0)

def validate_migration(source_url, source_token, target_url, target_token):
    """Compare configuration counts between tenants."""
    schemas = [
        "builtin:management-zones",
        "builtin:tags.auto-tagging",
        "builtin:alerting.profile",
        "builtin:monitoring.slo"
    ]

    results = []
    for schema in schemas:
        source_count = count_settings(source_url, source_token, schema)
        target_count = count_settings(target_url, target_token, schema)

        results.append({
            "schema": schema,
            "source": source_count,
            "target": target_count,
            "match": source_count == target_count
        })

    return results
```

Troubleshooting Common Issues

Issue	Cause	Solution
Missing configs	Schema not exported, or platform types skipped for lack of platform credentials	Export the specific schema; check the download log for `Skipped downloading` warnings
Deploy errors	Entity ID invalid	Map IDs to the target (§3 Option 2), or use an entity selector where the schema accepts one
Dashboard empty	Data not migrated	Expected (historic data doesn't migrate)
Alerts not firing	Credential issues	Re-enter webhook credentials
Synthetic failing	Location mismatch	Update location IDs

---

7. Summary

Migration Best Practices

Practice	Description
Plan thoroughly	Document everything before starting
Export first	Keep a backup of source config
Validate often	Check counts after each step
Test in dev	Validate in non-production first
Document exceptions	Note what needed manual work

Quick Reference: Migration Commands

Task	Monaco Command
Download all	`monaco download --manifest manifest.yaml --environment <env> --output-folder ./export`
Download platform types	`monaco download --manifest manifest.yaml --environment <env> --only-slo-v2 --only-segments --only-automation --only-documents` (needs `auth.platformToken` or `auth.oAuth`)
Download specific (classic inventory)	`monaco download --manifest manifest.yaml --environment <env> --settings-schema builtin:management-zones` (`--api` is for classic Configuration APIs only)
Validate	`monaco deploy manifest.yaml --dry-run` — Monaco ships no standalone `validate` subcommand (see AUTOM-03 §5)
Dry run	`monaco deploy manifest.yaml --dry-run`
Deploy	`monaco deploy manifest.yaml`

Series So Far

AUTOM-09 (Terraform GitOps setup recipe) and the appendix LABs (95–98) follow. Here's what the main sequence covered:

Notebook	Key Takeaway
AUTOM-01	Choose the right tool for your use case
AUTOM-02	Settings API is the foundation
AUTOM-03	Monaco for GitOps-style management
AUTOM-04	Terraform for state management
AUTOM-05	Workflows for event-driven automation
AUTOM-06	SDKs for custom applications
AUTOM-07	CI/CD for automated deployments
AUTOM-08	Migration patterns and validation

Additional Resources

- Monaco Documentation
- Terraform Provider
- Dynatrace API Reference
- Terraform export utility (DT docs)
- Terraform migration guide (DT docs)
- Migrate configuration — Managed to SaaS (DT docs)
- **M2S series** — for Managed-to-SaaS specific guidance, including the **M2S-95 LAB** Terraform migration walkthrough
- **DTM series** — the Managed source cluster: run the DTM-99 health-check checklist before exporting configuration from it
- Monaco commands reference (DT docs)

> **Key Takeaway:** Successful migration requires planning, the right tools, and thorough validation. Use Monaco or Terraform for bulk export/import, but always plan for manual work on credentials and entity references.

---

*Next: **AUTOM-09: Terraform GitOps Setup Recipe** for the operational architecture of standing up a Terraform GitOps shop (repo layout, state backends, lifecycle protections, team onboarding). For Managed-to-SaaS migrations, see the **M2S series**.*

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.